

PathFinder — Solution Documentation

HCLTech AMPlified Season 1 · Round 2

Team: PathFinder

Repository: https://github.com/shriansh1625/hcl_pathfinder

Date: August 2026

1. Executive Summary

PathFinder is an **evidence-driven career intelligence system** that turns a learner's natural-language goal into a dependency-aware learning route, diagnoses competency gaps against a fixed role ontology, and **adapts that route when new evidence changes the diagnosis**.

Unlike course recommenders that match keywords to playlists, PathFinder answers four questions every learner actually has:

1. **What am I missing** for my target role?
2. **Why this resource now** — not earlier, not later?
3. **What is proven** versus still unknown?
4. **How should my plan change** when I pass, fail, or struggle?

Core loop:

GOAL → CAREER → EVIDENCE → DIAGNOSIS → GAP → RECOMMENDATION → ACTION → NEW EVIDENCE → **ADAPTATION**

Signature moment: Assessment or progress feedback updates evidence → **Path V1 becomes Path V2** → completed work stays frozen → learner sees forensic "what changed and why."

2. Problem Statement

2.1 Industry gap

Most career platforms optimize for **content discovery**, not **competency diagnosis**:

Typical platform	Learner experience
Keyword search	"Here are 200 ML courses."
Completion %	"You are 40% ready" (often meaningless).

Typical platform	Learner experience
Static playlists	Same path for everyone targeting “Data Scientist.”
Black-box AI	Chatbot suggests courses with no audit trail.

Learners cannot see **prerequisite blockers**, **honest unknowns**, or **why the plan changed** after they prove a skill.

2.2 PathFinder’s thesis

Evidence → Diagnosis → Adaptation

PathFinder maintains a **learner competency model** from fused evidence, compares it to a **canonical role ontology**, sequences resources with **dependency awareness**, and **versions paths** when diagnosis shifts.

The LLM **explains** grounded facts. It does **not** rewrite proficiency, gaps, ranking, eligibility, sequencing, or adaptation.

3. Solution Overview

3.1 What PathFinder delivers

Capability	Description
Natural-language goal intake	Learner describes career intent in prose; system resolves to ontology-backed roles (or surfaces ambiguity / unsupported goals with recovery).
Multi-source evidence fusion	Self-report, assessments, and progress feedback combine into per-skill proficiency with confidence.
Role-relative gap engine	Gaps, blockers, and interventions computed against fixed career targets — not generic percentages.
Prerequisite-aware path	Resources sequenced with EXECUTABLE / WAITING / BLOCKED semantics.
Assessment loop	Canonical gates update evidence and can trigger path adaptation.
Progress loop	Complete / struggled / skip with optional confidence feeds evidence without inventing levels.

Capability	Description
Path versioning	V1 immutable; V2 created on material diagnosis change; completed items frozen.
Forensic transparency	WHY drawer, "What changed," History timeline, Skill Map, Judge Mode.
Grounded AI (optional)	Natural-language explanations over verified backend facts only.

3.2 Ontology (source of truth)

All intelligence is grounded in versioned YAML under `data/` :

- 8 canonical careers (e.g. AI/ML Engineer, Cybersecurity Analyst, Data Engineer)
- 47 skills with proficiency targets per role
- 58 dependency relationships
- 62 catalogued resources
- 4 canonical assessments

The ontology is **never extended by the LLM**. Unknown learner phrases resolve to RESOLVED, AMBIGUOUS, or UNSUPPORTED — never invented roles.

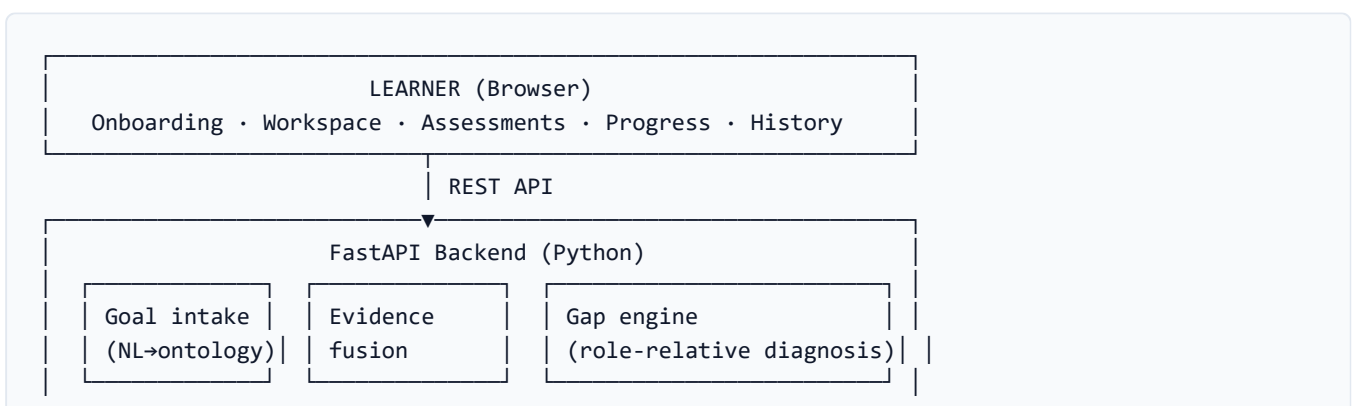
3.3 UNKNOWN semantics (design differentiator)

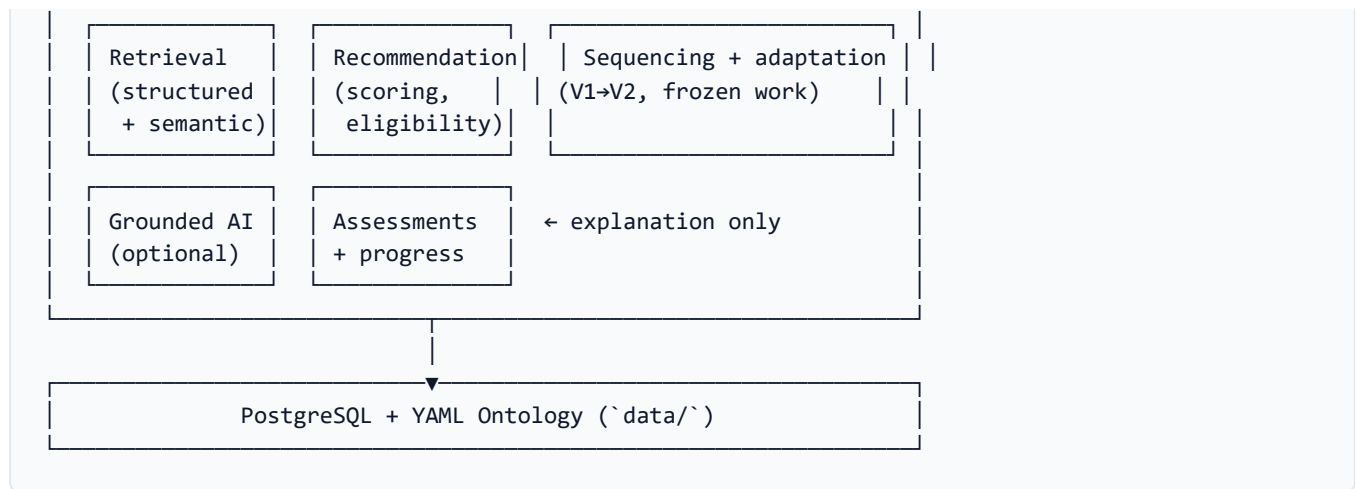
UNKNOWN means "no evidence yet" — not zero percent.

Treating missing evidence as 0% would falsely diagnose gaps and pollute recommendations. PathFinder shows **NO EVIDENCE** states until the learner or an assessment supplies proof.

4. System Architecture

4.1 High-level diagram





4.2 Technology stack

Layer	Technology
Frontend	Next.js 15, React, TypeScript, Tailwind CSS
Backend	FastAPI, SQLAlchemy, Alembic
Database	PostgreSQL 16
Semantic ML (optional)	fastembed / BGE embeddings (local, no external API)
LLM (optional)	Groq GPT-OSS-120B via OpenAI-compatible API — explanation + goal language layer only
CI	GitHub Actions (pytest, vitest, build, benchmark)

4.3 AI/ML boundaries (critical for judges)

Component	LLM involved?	Authority
Goal language interpretation	Optional (Groq)	Ontology resolver is final
Evidence fusion	No	Deterministic
Gap engine	No	Deterministic
Recommendation scoring	No (+ optional 5% semantic)	Deterministic
Path sequencing	No	Deterministic
Assessment scoring	No	Deterministic
Adaptation / PathDiff	No	Deterministic
Grounded explanations	Optional	Facts from backend; validator enforced

Verified facts → Grounded AI → Explanation. Never the reverse.

5. Key Features (Judge-Visible)

5.1 Natural-language goal intake

Learners type goals like “I want to be a penetration tester” or “I want a career in data.”

Resolution outcomes:

- **RESOLVED** — maps to canonical role (e.g. pen tester → Cybersecurity Analyst)
- **AMBIGUOUS** — learner chooses between supported candidates (e.g. Data Engineer vs Data Analyst)
- **UNSUPPORTED** — honest recovery; manual career selection always available

Never stuck, blank, or infinite loading.

5.2 Dashboard (KNOW + DIAGNOSE)

- Destination role and goal text
- Gap summary with honest UNKNOWN handling
- Blockers with prerequisite chains
- Next recommended action from backend

5.3 My Path (sequenced route)

- Week-grouped resources with blocker semantics
- **WHY THIS RESOURCE** drawer — backend factors: gap, role relevance, prerequisites, intervention
- Progress actions: Complete / Struggled / Skip

5.4 Assessments (PROVE)

- Canonical skill gates tied to ontology
- Submission updates competency model
- Result screen shows diagnosis shift

5.5 Adaptation (ADAPT) — signature UX

- **Path V1** — initial sequenced route
- New evidence → **Path V2** with FLIP animation
- **Frozen completed work** never reordered away
- **What changed** — added / removed / moved / blocked items with reasons

- **Why this changed** — forensic adaptation trace
- **History** — version timeline

5.6 Skill Map

- Dependency visualization: blockers, dependents, preparation skills
- Select a skill to illuminate neighborhood

5.7 Judge Mode

- Guided tour aligned to evaluation criteria
- Surfaces proof points without breaking flow

5.8 Dual theme

- Dark (atmospheric) and Light (warm parchment) themes
- No theme flash; accessibility audited on 13 pages per theme

6. Innovation & Differentiation

Innovation	Why it matters
Versioned paths with frozen work	Learners don't lose credit when the plan adapts.
PathDiff + FLIP	Adaptation is visible, not a silent rerank.
UNKNOWN honesty	No fake proficiency from missing data.
Blocker-aware sequencing	Explains "why can't I start?"
Forensic WHY	Every recommendation is auditable.
Second-learner personalization	Same role + different evidence → different path (proven).
Failure-safe intake	NL goals never dead-end; ontology is authoritative.
Bounded semantic signal	ML assists retrieval (5% cap), doesn't control diagnosis.

7. Verification & Quality Evidence

Gate	Result	Artifact
Backend tests	237 passed	<code>python -m pytest -q</code>
Frontend tests	56 passed	<code>npm test</code>
Production build	PASS	<code>npm run build</code>
Intelligence benchmark	20/20	<code>artifacts/intelligence_benchmark.json</code>
Failure matrix	9/9	<code>artifacts/failure-matrix/summary.json</code>
Multi-career isolation	3/3 unique paths	<code>artifacts/multi-career-proof/proof.json</code>
Second-learner personalization	PASS	<code>artifacts/second-learner-proof/proof.json</code>
Accessibility	0 critical / 0 serious	<code>artifacts/accessibility/summary-dark.json</code>
API smoke	19/19	<code>scripts/api_smoke_test.py</code>
Mobile overflow (Path V2)	0px	<code>artifacts/final-mobile-overflow/summary.json</code>
Goal intake browser	15/15	<code>artifacts/goal-intake-qa/summary.json</code>

8. Deployment & Reproducibility

8.1 Local setup (judge flow)

Requirements: Python 3.11+, Node 20+, Docker.

```
git clone https://github.com/shriansh1625/hcl_pathfinder.git
cd hcl_pathfinder
cp .env.example .env.local
cp frontend/.env.example frontend/.env.local

docker compose up -d db
cd backend && pip install -r requirements.txt && alembic upgrade head && cd ..
python scripts/validate_ontology.py && python scripts/seed.py

# Terminal 1 – API
cd backend && uvicorn app.main:app --host 0.0.0.0 --port 8000
```

```
# Terminal 2 – UI (production mode)
cd frontend && npm install && npm run build && PORT=3002 npm run start
```

Open <http://localhost:3002> → use **Judge demo** (~90s) for fastest evaluation path.

8.2 Cloud deploy (optional)

See [docs/DEPLOY_VERCEL_RENDER.md](#) — Render (API + Postgres) + Vercel (frontend). Groq optional for grounded AI.

8.3 Environment variables

Variable	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection
<code>NEXT_PUBLIC_API_URL</code>	Frontend → API (required for build)
<code>PATHFINDER_CORS_ORIGINS</code>	Allowed frontend origins
<code>PATHFINDER_AI_API_KEY</code>	Optional Groq key — never commit
<code>PATHFINDER_SEMANTIC_ENABLED</code>	Optional local embeddings

9. Security & Privacy

- API keys **backend-only** via environment variables
- `.env.local` gitignored; no secrets in repository or artifacts
- LLM receives delimited untrusted user text; cannot create ontology entities
- Append-only evidence audit trail
- CI runs with `PATHFINDER_AI_PROVIDER=stub` — no external LLM required

10. Known Limitations (Honest)

1. **No permanent hosted demo** — judges run locally per setup instructions
2. **Stub AI default** — architecture supports Groq; explanations use deterministic fallback without key
3. **Mobile path** — compact compass layout; desktop shows full spatial route
4. **Ontology scope** — 8 careers; unsupported goals require manual selection or future ontology expansion

11. Team & Repository

Item	Value
Repository	https://github.com/shriansh1625/hcl_pathfinder
Branch	<code>main</code>
License	MIT
Documentation	docs/ — Architecture, Judge FAQ, Proof Closure, Goal Intake Hardening
Screenshots	artifacts/submission-screenshots/

12. Conclusion

PathFinder demonstrates a **complete adaptive learning intelligence loop** with auditable deterministic engines, honest evidence semantics, visible path versioning, and production-grade verification. It is designed for judges who ask “**show me the proof**” — not just the pitch.

Evidence → Diagnosis → Adaptation.